Semantic tagging

FDD — Native Fault Detection & Diagnostics for T3000 · built Nov 13, 2026

FDD — Native Fault Detection & Diagnostics for T3000

> Native Rust FDD engine with DB-managed rules (no SQL engine, nothing embedded).

1. Overview & goal

Add a **Fault Detection & Diagnostics (FDD)** capability to T3000 so the platform can proactively detect common HVAC equipment faults from historical trend data, and let the AI chat explain them.

The design deliberately follows open-fdd's **concepts** (rules as tunable data, semantic roles, confirm windows) but re-implemented **natively in T3000's Rust stack**:

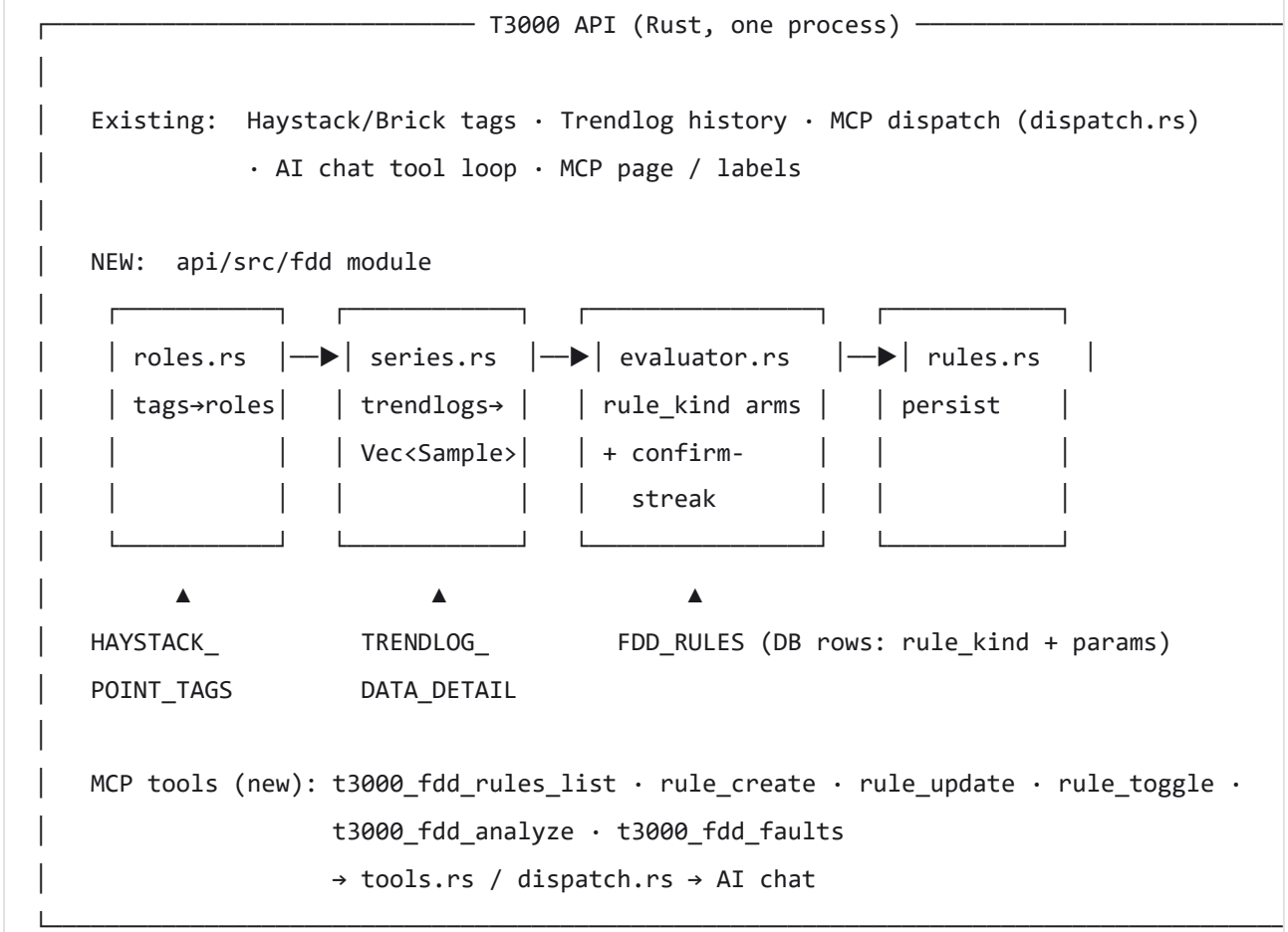
- ✅ rules stored as **DB rows** (managed like the existing auto-tagging rules)
- ✅ one reusable **Rust evaluator** (no SQL engine, no DataFusion)
- ✅ nothing compiled into the binary (rules are data)
- ✅ reuses what T3000 already has: Haystack/Brick tags, trendlog history, MCP server, AI chat

2. Design decisions

Decision	Choice	Why
Engine	**Native Rust evaluator**	no SQL-in-DB, no DataFusion, typed, testable
Rule storage	**DB table `FDD_RULES`**	managed at runtime like `HAYSTACK_AUTO_TAGGING_RULES`; nothing embedded
Role source	**Haystack/Brick tags**	already in T3000 (`HAYSTACK_POINT_TAGS` , `HAYSTACK_POINT_BRICK_CLASS`)
Time-series	**TRENDLOG_DATA_DETAIL**	existing history; pivot to wide samples on demand
Discovery	**not built**	explicit `analyze` calls; scheduled scans are a later phase
Findings	**`FDD_FINDINGS` table**	`t3000_fdd_analyze` persists active faults; `t3000_fdd_faults` lists history

3. Architecture

TEXT



4. Data model — `FDD_RULES` table

```SQL
CREATE TABLE IF NOT EXISTS FDD_RULES (
  id             INTEGER PRIMARY KEY AUTOINCREMENT,
  rule_id        TEXT NOT NULL UNIQUE,    -- 'ECON-4'
  rule_name      TEXT NOT NULL,
  category       TEXT,                    -- 'economizer' | 'sensor' | 'vav' | 'chw' | 'z
  description    TEXT,
  rule_kind      TEXT NOT NULL,           -- which Rust evaluator runs
  required_roles TEXT,                    -- JSON array of semantic roles
  params_json    TEXT,                    -- JSON of tunable params
  severity       TEXT DEFAULT 'warning', -- info | warning | critical
  enabled        BOOLEAN DEFAULT 1,
  created_at     TEXT,
  updated_at     TEXT
);
```

> `FDD_FINDINGS` stores persisted fault detections — written by `t3000_fdd_analyze` and read by `t3000_fdd_faults`. Time-series and tags reuse existing tables.

Seed

The catalog is seeded by the SeaORM migration `m20260819_add_fdd_tables` (fresh DBs) and by `fdd::rules::ensure_schema` on startup (idempotent `INSERT OR IGNORE`, so existing DBs pick up new rules). After seeding the DB is authoritative — rules can be edited/toggled/deleted at runtime with no rebuild.

```SQL
INSERT INTO FDD_RULES (rule_id, rule_name, category, description, rule_kind, required_ro
VALUES
('ECON-4','Low outdoor-air fraction','economizer',
 'Economizer not bringing in enough OA when free-cooling is available',
 'EconomizerOaFraction',
 '["mat","rat","oa_t","fan_cmd"]',
 '{"oa_min_pct":15,"confirm_rows":4,"poll_seconds":300}',
 'warning', 1);
```

---

5. The Rust engine

5.1 Sample & roles

RUST

```rust
/// One "wide" history row built from TRENDLOG_DATA_DETAIL for a device/equipment.
/// Roles are resolved from Haystack tags; values are keyed by role.
pub struct Sample {
    pub ts: String,                      // LoggingTime_Fmt, e.g. "2026-08-19 12:30:00"
    pub values: HashMap<String, f64>,    // role → value, e.g. "oa_t" → 55.0
}
```

`roles.rs` maps each point's **Haystack tags / Brick class** to a role on `Sample`:

TEXT

```
OA_T        → oa_t       MAT         → mat         SupplyTemp → sat
RAT         → rat        ZONE_T      → zone_t      FAN_CMD    → fan_cmd
FAN_STATUS  → fan_status OA_DAMPER   → damper_pct  CHW_S      → chw_s
CHW_R       → chw_r      CHW_DP      → chw_dp      SAT_SP     → sat_sp
```

5.2 The reusable evaluator (the heart)

Every rule = **a per-sample condition** + **a confirm streak** + **a sample interval** → fault hours.

RUST

```rust
/// Generic confirm-window evaluator: count fault only after N consecutive samples.
fn fault_hours<F>(series: &[Sample], mut raw_fault: F, confirm_rows: u32, poll_seconds:
where F: FnMut(&Sample) -> bool
{
    let mut streak = 0u32;
    let mut hours = 0.0;
    for s in series {
        streak = if raw_fault(s) { streak + 1 } else { 0 };
        if streak >= confirm_rows {
            hours += poll_seconds as f64 / 3600.0;
            streak = 0;
        }
    }
    hours
}

/// Read a role's value from a sample (None if the device has no such role).
fn field(s: &Sample, role: &str) -> Option<f64> { s.values.get(role).copied() }

/// Dispatch a FDD_RULES row to the right condition. Returns fault hours + evidence.
pub fn eval_rule(rule: &Rule, series: &[Sample]) -> Finding {
    let confirm = rule.params["confirm_rows"].as_u64().unwrap_or(4) as u32;
    let poll = rule.params["poll_seconds"].as_u64().unwrap_or(300);
    match rule.rule_kind.as_str() {
        "ThresholdAbove" => {
            let f = rule.params["field"].as_str().unwrap_or("");
            let limit = rule.params["limit"].as_f64().unwrap_or(0.0);
            let hours = fault_hours(series, |s| field(s, f).is_some_and(|v| v > limit),
            Finding { fault_hours: hours, evidence: json!({"field": f, "limit": limit})
        }
        "ThresholdBelow" => { /* same, reversed */ }
        "RangeBand"      => { /* value outside [lo, hi] */ }
        "StuckValue"     => { /* no change over a sliding window of `window_rows` (froze
        "FanMismatch"    => { /* (fan_cmd on) != (fan_status on) after 0..100 normalisat
        "EconomizerOaFraction" => { /* OA fraction (mat-rat)/(oa_t-rat) < oa_min_pct whi
        "EconomizerStuckClosed" => { /* damper_pct < 5 while fan on and oa_t < mat */ }
        "ChwLowDeltaT"   => { /* chw_r - chw_s < min_dt */ }
        "SupplyTempDeviation" => { /* |sat - sat_sp| > max_dev */ }
        _ => Finding { fault_hours: 0.0, evidence: json!({}) },  // new rule_kinds = new
```

```
    }
}
```

Adding a rule = one new `rule_kind` arm **or** reuse a generic one + insert a `FDD_RULES` row.

---

6. The complete rule catalog

Starter catalog (16 rules). All are DB rows seeded on first run; every rule is tunable via `params_json` and toggleable via `enabled`.

6.1 Economizer

rule_id	Name	rule_kind	Required roles	Default params	Severity
ECON-1	OA damper stuck closed	`EconomizerStuckClosed`	`oa_t, mat, fan_cmd, damper_pct`	`confirm_rows:4, poll_seconds:300`	warning
ECON-3	Mechanical cooling without economizing	`EconomizerOaFraction`	`mat, rat, oa_t, fan_cmd`	`oa_min_pct:0, confirm_rows:4`	warning
ECON-4	Low OA fraction	`EconomizerOaFraction`	`mat, rat, oa_t, fan_cmd`	`oa_min_pct:15, confirm_rows:4`	warning
ECON-6	Economizer freezing risk	`ThresholdBelow`	`mat`	`field:"mat", limit:2.0, confirm_rows:3`	critical
ECON-7	Not economizing when it should	`EconomizerOaFraction`	`mat, rat, oa_t, fan_cmd`	`oa_min_pct:0, confirm_rows:4`	info

6.2 Fan

rule_id	Name	rule_kind	Required roles	Default params	Severity
CMD-1	Fan cmd/status mismatch	`FanMismatch`	`fan_cmd, fan_status`	`confirm_rows:4, poll_seconds:300`	warning
FAN-RUNTIME	Fan runtime hours (metric)	`ThresholdAbove`	`fan_cmd`	`field:"fan_cmd", limit:0.05`	info

6.3 Supply air sensor

rule_id	Name	rule_kind	Required roles	Default params	Severity
SAT-HIGH	Supply air temp too high	`ThresholdAbove`	`sat`	`field:"sat", limit:100, confirm_rows:4`	critical
SAT-LOW	Supply air temp too low	`ThresholdBelow`	`sat`	`field:"sat", limit:40, confirm_rows:4`	critical
SAT-DEV	Supply air temp deviates from setpoint	`SupplyTempDeviation`	`sat, sat_sp`	`max_dev:5, confirm_rows:4`	warning
SAT-STUCK	Supply air temp sensor frozen	`StuckValue`	`sat`	`deadband:0.1, window_rows:12`	warning

6.4 Zone / VAV

rule_id	Name	rule_kind	Required roles	Default params	Severity
VAV-1	Zone comfort band violation	`RangeBand`	`zone_t`	`lo:70, hi:75, confirm_rows:4`	warning
ZONE-STUCK	Zone temp sensor frozen	`StuckValue`	`zone_t`	`deadband:0.1, window_rows:12`	warning

6.5 Chilled water

rule_id	Name	rule_kind	Required roles	Default params	Severity
CHW-1	Low delta-T across coil	`ChwLowDeltaT`	`chw_s, chw_r`	`min_dt:5, confirm_rows:4`	warning
CHW-2	CHW supply pressure low	`ThresholdBelow`	`chw_dp`	`field:"chw_dp", limit:8, confirm_rows:4`	warning
CHW-3	CHW supply temp out of band	`RangeBand`	`chw_s`	`lo:40, hi:48, confirm_rows:4`	warning

> The catalog grows by **inserting rows** (and, rarely, adding a `rule_kind` evaluator arm).

---

7. Management (same as auto-tagging)

Action	MCP tool	DB effect	Like
List rules	`t3000_fdd_rules_list`	`SELECT * FROM FDD_RULES`	`t3000_haystack_list_rules`
Create custom rule	`t3000_fdd_rule_create` (confirm)	`INSERT`	`t3000_rule_create`
Edit rule / tune params	`t3000_fdd_rule_update` (confirm)	`UPDATE params_json`	—
Enable / disable	`t3000_fdd_rule_toggle` (confirm)	`UPDATE enabled`	`t3000_rule_toggle`
Backup / share	`t3000_fdd_rule_export` → JSON	read	—
Restore	`t3000_fdd_rule_import` (confirm)	upsert	—

Tool input example — `t3000_fdd_rule_toggle`:

```JSON
{ "rule_id": "ECON-4", "enabled": false, "confirm": true }
```

Tool input example — `t3000_fdd_analyze`:

```JSON
{
  "serial_number": 123,
  "equipment": "AHU-1",
  "range_hours": 24,
  "rules": ["ECON-4", "SAT-HIGH"]
}
```

---

8. Findings output

```JSON
{
  "device": 123,
  "equipment": "AHU-1",
  "range_hours": 24,
  "findings": [
    {
      "rule_id": "ECON-4",
      "rule_name": "Low outdoor-air fraction",
      "severity": "warning",
      "fault_hours": 1.5,
      "evidence": { "oa_min_pct": 15, "samples": 288, "min_oa_frac": 3.2 },
      "suggestion": "Check OA damper actuator and economizer control."
    }
  ]
}
```

---

9. AI chat integration

`t3000_fdd_*` tools join `TOOLS` → `get_all_tool_defs()` automatically. A user asks:

> "Why is AHU-1 faulting?"

The AI calls `t3000_fdd_analyze {serial_number:123, equipment:"AHU-1", range_hours:24}` → gets findings → explains in plain language + suggests fixes (optionally via `t3000_doc_read` / `t3000_device_diagnostics`).

Frontend additions (same pipeline as existing tools):

- `useAiChatStream.ts` tool labels (e.g. `t3000_fdd_analyze: 'Running fault detection…'`)
- `McpServerPage.tsx` tool-table rows + Examples prompts

---

10. Testing

- **Unit** (no DB): build synthetic `Vec<Sample>`, run `eval_rule` per `rule_kind`, assert `fault_hours` (streak/confirm behaviour).
- **Integration** (MCP test suite, like `device_operations.rs`): tool existence + schema; DB test seeds `FDD_RULES`, runs `t3000_fdd_analyze` against a small synthetic series inserted into `TRENDLOG_DATA_DETAIL`.
- **Tool-count** test: bump the count in `tool_definitions.rs` when tools are added.

---

11. Known limitations

- **Tag coverage** — rules only fire where auto-tagging produced the required roles. `t3000_fdd_analyze` returns `roles_found` per analysis so gaps are visible.
- **Trendlog resolution** — low-resolution logs limit detection quality; `poll_seconds` in each rule must match the actual trendlog log interval for accurate fault hours.
- **Rule correctness** — the starter catalog is heuristic; thresholds should be validated and tuned against real site data.